

HelixMemory

HelixMemory

AIエージェントのための記憶脳——四つの最高峰エンジンを融合

概要

HelixMemoryは、Go SDKであり、四つの先進的な記憶システム（Mem0、Cognee、Letta、Graphiti）を統合し、単一の認知記憶エンジンとして並列検索と結果の融合を行います。AIアプリケーションにとって、四つの断片化された記憶層の代わりに、耐久性があり重複排除され再ランク付けされた一つの記憶層を提供します。

短い説明

HelixMemoryは、Go SDKであり、Mem0、Cognee、Letta、GraphitiをAIアプリケーションのための統合認知記憶エンジンに融合します。書き込みはインテリジェントにルーティングされ、全てのバックエンドを並列検索し、三段階の収集・重複排除・再ランク付けパイプラインを通じて結果を融合します。

長い説明

HelixMemoryは、AIアプリケーションのための統合認知記憶エンジンであり、Go SDK（モジュール `digital.vasic.helixmemory`、Go 1.25以降）として提供されます。その根底にある考え方は、単一の記憶プロジェクトが全てにおいて最高の性能を発揮することは決してないというものです。そのため、ゼロから記憶を再実装して一つのプロジェクトの盲点を引き継ぐのではなく、四つの最高峰システムを統合し、それぞれの強みを活かすことを選びました。具体的には、Mem0は動的事実抽出と嗜好管理、CogneeはECLパイプラインで構築された意味論的知識グラフ、Lettaは編集可能な記憶ブロックとスリープ時計算を備えた状態保持型エージェントランタイム、Graphitiは事実の時間的変化を推論する二時制知識グラフを担当します。

これら四つの独立したストアを一つの脳に変えるのが融合エンジンです。書き込み時には、入力された記憶は内容に応じて最適なバックエンドにルーティングされます。読み込み時には、クエリが全てのバックエンドに並列で送信され、得られた結果は三段階の融合パイプライン——収集、重複排除、クロスソース再ランク付け——を経て、呼び出し側には四つの雑多で重複した結果セットではなく、一つの整理されたランク付け済み回答が返されます。各バックエンドはサーキットブレーカーで保護されており、一つのエンジンがダウンしてもブレーカーが作動し、残りのバックエンドがサービスを継続するため、記憶層全体が停止することはありません。エンジンはドロップイン可能なMemoryStoreインターフェースを実装しているため、単純な記憶プロバイダーの直接置き換えとして機能し、呼び出し側のアーキテクチャ変更は不要です。また、Prometheusメトリクスにより、ルーティングや融合の内部動作が完全に可視化されます。

HelixMemoryは、HelixAgent——より広範なHelix AIアンサンブル——のための記憶層として開発されました。このシステムは、家族の「ブラフ排除テスト」の原則を記憶領域にも適用しています。プロセス内チャレンジランナーが実際の本番コードパス——ルーティング、融合、トランスレーター、サーキットブレーカー——を実行し、ペアミューテーションラッパーが意図的に不変条件を反転させて、ロジックが壊れた際にテストが実際に失敗することを証明します。これにより、テストスイートがグリーンであれば、それは確かな信頼性を意味します。

なぜこれを作ったのか

コンテンツ

AIエージェントには長期的で高品質な記憶が必要だが、そのエコシステムは分断されている。各メモリプロジェクト（Mem0、Cognitive、Letta、Graphiti）はそれぞれ一つの強みを持ちながら、他の部分では弱点を抱えている。HelixMemoryは、HelixAgentに対して、これらの強みを統合しつつ、どれか一つにロックインされることのない単一のメモリ基盤を提供するために開発された。

なぜゲームチェンジャーなのか

これまでの選択の強制を終わらせる。通常は同じスロットを争う四つのメモリシステムが、単一のインターフェースの背後で補完的なバックエンドとして機能するようになる。これにより、アプリケーションは動的な事実抽出、意味論的知識グラフ、状態を持つエージェントメモリ、二時制推論を同時に利用できるようになり、重複排除やクロスソースの再ランキングも自動的に処理される。従来は非現実的だった「どのメモリエンジンを採用するか」という偽のジレンマを解消する。HelixMemoryは、一つのドロップイン可能なMemoryStoreの背後で、すべてのエンジンの強みを同時に享受でき、どのエンジンの弱点も引き継がず、ロックインのリスクも負わない。

革新的な点

- 複数バックエンドの融合（収集→重複排除→クロスソース再ランキング）により、単一のストアに縛られることなく、一つのランキング結果セットを返す。
- インテリジェントな書き込みルーティングにより、各メモリの内容を分類し、最適なエンジンに送信。適切なデータが適切なストアに格納される。
- バックエンドごとのサーキットブレーカーによるグレースフルデグラデーション—障害が発生したエンジンは隔離され、致命的な障害にはならず、残りのエンジンがサービスを継続。
- アイドル時のコンピュート統合（Lettaを介して）により、クエリ時だけでなく、アイドル期間中にメモリを再構築。
- ブラフ防止検証：本番コードに対するチャレンジランナーと、不変条件が反転した際に必ずテストが失敗するミュレーションラッパーを組み合わせ、テストゲートが単なるトートロジーではなく、実際の検証であることを証明。

最大の技術的課題とその解決策

- 四つの異種バックエンドを一つの一貫した結果セットに統合—各エンジンは独自の形式でメモリを返し、単純にマージすると重複や比較不可能なランキングが発生。解決策は、型付き融合エンジンを用いてソース間で収集し、重複を排除し、共通の基準で再ランキング。マージ時に結果が密かに欠落したり二重カウントされたりしないよう、融合カウントの不変条件をテストで検証。
- バックエンドがダウンしてもシステム全体が停止しない—一つのメモリエンジンが到達不能になっても、レイヤー全体が停止してはならない。解決策は、バックエンドごとのサーキットブレーカーを導入し、障害閾値に達するとクローズド→オープン（障害後）→ハーフオープン（タイムアウト後）の状態遷移を行い、障害のあるバックエンドを隔離しつつ、健全なバックエンドからサービスを継続。
- メモリロジックが実際に機能することを証明する、単なるコンパイル成功以上の検証—テストスイートがグリーンでも、テストが失敗しなければ意味がない。解決策は、本番コード（ルーティング、融合、トランスレータ、サーキットブレーカー）を駆動するインプロセスのチャレンジランナーと、不変条件を反転させてテストがレッドになることを要求するミュレーションラッパーを組み合わせ、ゲートが検証可能なものであり、トートロジーではないことを証明。

テックスタック

- **Go**（バージョン1.25以降） — SDKとランタイムを統合したもの。4つのバックエンドにまたがる並列読み込みのファンアウトが並行処理の課題となる中、Goのgoroutineにより低コストで実現可能であり、そのインターフェースタイプによってシステム全体にクリーンな境界（MemoryStore）を提供し、呼び出し側が依存できる構造を確立している。

- **Mem0** — 動的な事実抽出と嗜好管理を行うバックエンド。ユーザーの実際の嗜好や浮上した事実を扱う「記憶の断片」を担当。
- **Cognee** — ECLパイプラインを基盤としたセマンティック知識グラフバックエンド。単なる事実の羅列ではなく、構造化された関連知識を保持する。
- **Letta** — 編集可能なメモリブロックとスリープ時の計算処理を備えたステートフルなエージェントランタイムバックエンド。エージェントの状態としてメモリを永続化し、アイドル期間中に統合する必要がある場合に使用。
- **Graphiti** — 二時制知識グラフバックエンド。事実や関係性の現在の値だけでなく、時間の経過による変化を推論するために用いる。
- **PostgreSQL + Neo4j + Redis** — バックエンドが実際に稼働する本番相当のデータストア。make infra-startによる統合テスト環境として構築されており、モックではなく実インフラ上でスイートが動作する。
- **Prometheus** — メトリクスとオブザーバビリティをフュージョンパイプラインに組み込み、ルーティングやフュージョンの挙動を本番環境で可視化。ブラックボックス化を防ぐ。
- **i18n** 翻訳レイヤー — 名前空間 (helixmemory_) を持つ文字列インターフェースを維持。将来的にユーザー向けレイヤーをローカライズする際、コア部分を改修することなく対応可能。

ステータスと率直な注意点

- ステータス：ベータ版。動作するSDK。HelixAgentのメモリレイヤーとして構築。
- ライセンス：未定。GitHubのAPIによるライセンス検出が行われておらず、未確認／未宣言。
- 表示名「HelixMemory」はリポジトリmemoryに対応。READMEに記載された精度数値はアップストリームベンダーの主張であり、HelixMemoryによる実測値ではないため、ここでは省略。

優先度区分： Helix-プライマリ。